
Kairos

<Test Plan>

Version <1.0>

	Version: <1.3>
Test plan	Fecha: <14/12/2025>
<Test plan AlbaGym SoftwareOS>	

Revisions

Date	Version	Description	Author
14/12/2025	1.3	Final version	Marcos Labrador, Lida Lopez, Adrian sevilla

	Version: <1.3>
Test plan	Fecha: <14/12/2025>
<Test plan AlbaGym SoftwareOS>	

Index

1. Test plan identifier	1
2. Introduction	1
3. Test items	1
4. Features to be tested	2
5. Features not to be tested	2
6. Approach	2
7. Item pass/fail criteria	2
8. Suspension criteria and resumption requirements	2
9. Test deliverables	3
10. Testing tasks	3
11. Environmental needs	3
12. Responsibilities	3
13. Staffing and training needs	4
14. Schedule	4
15. Risks and contingencies	4
16. Approvals	4

	Version: <1.3>
Test plan	Fecha: <14/12/2025>
<Test plan AlbaGym SoftwareOS>	

Test plan

To prescribe the scope, approach, resources, and schedule of the testing activities. To identify the items being tested, the features to be tested, the testing tasks to be performed, the personnel responsible for each task, and the risks associated with this plan.

1. Test plan identifier

TestPlan_AlbaGym_SoftwareOS_2025

2. Introduction

2.1 Software Items and Features to be Tested

The object of testing is the **AlbaGym Application**, a system designed to manage gym activities, user purchases, and inventory management. This system has an API and a web service that also will be tested.

Based on the functional specifications of the different use cases, the following software items and features will be verified:

- **Create planning use case**
 - **Create Planning:** Testing the end-to-end flow of creating a workout plan, including filtering classes by type/date, selecting classes, and validating mandatory inputs (Name, Weeks, Payment Method).
 - **Validation Logic:** Verifying system warnings for lack of class availability, past dates, empty selection, and capacity limits.
 - **Data Persistence:** Verifying the correct storage of Plan and PlanItem entities.
- **Report incident use case**
 - **Report Incident:** Testing the submission of facility incidents, including filtering items by name/location and selecting affected items.
 - **Submission Integrity:** verifying mandatory fields (Title, Date, Exercise, Priority) are enforced and that the system correctly transitions the incident state to "submitted".
 - **Important note:** this case was planned in the initial implementation of the AlbaGym but for this specific implementation won't be testing nor created.
- **Purchase Item use case**
 - **Purchase Items:** Testing the purchasing flow, including item selection, filtering by name/brand, and cart management.

	Version: <1.3>
Test plan	Fecha: <14/12/2025>
<Test plan AlbaGym SoftwareOS>	

- **Stock & Payment:** Verifying validation logic that prevents purchasing more than the available quantity and ensures valid payment methods and delivery addresses are provided.
- **Transaction Record:** Verifying the creation of Purchase and PurchaseItem records.
- **Restock Item use case**
 - **Restock Items:** Testing the administrator's workflow to replenish stock, including filtering items and inputting mandatory restock details (Title, Address, Quantity).
 - **Business Rules:** Verifying logic that warns if QuantityAvailableForPurchase exceeds QuantityForRestock and ensures the RestockResponsible user is correctly associated.

2.2 Need for the Software

The basics software programs that are needed to run and test the AlbaGym application are:

- Azure
- Sql to local testing
- Visual studio code 2022
- Selenium
- API and WEB services

2.3 References

The following documents are referenced within this test plan:

- **Requirement Specification:** *Use Case - AlbaGym.pdf*
- **Testing Standard:** *ISO/IEC/IEEE 29119-2:2021* – Software Testing — Part 2: Test Processes.
- **Project Plan:** N/A
- **Quality Assurance Plan:** N/A
- **Configuration Management Plan:** N/A

3. Test items

References of the users stories.(dont copy and paste)

	Version: <1.3>
Test plan	Fecha: <14/12/2025>
<Test plan AlbaGym SoftwareOS>	

Identify the test items including their version/revision level. Also specify characteristics of their transmittal media that impact hardware requirements or indicate the need for logical or physical transformations before testing can begin (e.g., programs must be transferred from tape to disk).

Supply references to the following test item documentation, if it exists:

- a) Requirements specification;
- b) Design specification;
- c) Users guide;
- d) Operations guide;
- e) Installation guide.

Reference any incident reports relating to the test items.

The following items representing the core of the Kairos system will be tested. These items are located in repository.

Items that are to be specifically excluded from testing may be identified.

Software Item 1: Web API Module (Backend)

Version: 3.0 In the development branch.

Transmittal media: Hosted on GitHub Repository

Characteristics: Restful API built with .NET. It handles the business logic for Plans, Purchase, and Restocks.

Transformation: The source code must be compiled and deployed in Azure or run on localhost in Visual Studio.

Software Item 2: Razor Pages Web App (Frontend)

Version 1.0

Transmittal media: Hosted on GitHub Repository

Characteristics: User Interface allowing interaction with the system to enroll in classes, buy items, and manage restocks.

The following documentation is referenced to support the testing process:

Requirements specification:

Design specification The UML Class Diagrams and Architecture (Data models for Plan, Purchase and Restock entities).

User guide:

Installation guide:

Items Excluded:

Report Incident with Items for Exercise: This feature is excluded from this testing phase.

	Version: <1.3>
Test plan	Fecha: <14/12/2025>
<Test plan AlbaGym SoftwareOS>	

Non functional items: Database performance, third party NuGet packages, and security penetration testing.

4. Features to be tested

USE CASE PLAN

- ESC-2_TC1: No classes available
- ESC-3_TC2: Filter by date
- ESC-3_TC3: Filter by range of dates
- ESC-3_TC4: Filter by itemType
- ESC-3_TC5: Filter by itemType and date
- ESC-4_TC6: Date before today
- ESC-5_TC7: Modify plan selected
- ESC-6_TC8: No classes selected
- ESC-7_TC9: No filters
- ESC-7_TC10: Missing data
- ESC-8_TC11: Enough capacity

USE CASE PURCHASE

- ESC-1_TC1: BF
- ESC-2_TC2: No item available
- ESC-3_TC3: Filter by name
- ESC-3_TC4: Filter by brand
- ESC-4_TC5: Modify renting cart
- ESC-5_TC6: Wrong email/username
- ESC-5_TC7: Wrong description
- ESC-5_TC8: Wrong payment method (The user has only Bizum)
- ESC-5_TC9: Wrong street (Too short)
- ESC-5_TC10: Wrong City (Too short)
- ESC-5_TC11: Wrong Country (Too short)
- ESC-6_TC12: Quantity Available < Quantity Selected>

USE CASE RESTOCK

- ESC-1_TC1: BF
- ESC-3_TC2: Filter by name

	Version: <1.3>
Test plan	Fecha: <14/12/2025>
<Test plan AlbaGym SoftwareOS>	

- ESC-3_TC3: Filter by quantity
- ESC-2_TC4: Restock not available
- ESC-2_TC5: Restock not available
- ESC-2_TC6: Restock not available
- ESC-4_TC7: No items selected to restock
- ESC-5_TC8: No title provided
- ESC-5_TC9: No address provided
- ESC-5_TC10: Wrong Description
- ESC-5_TC11: Wrong restock response
- ESC-6_TC12: Modify restock cart
- ESC-6_TC13: Modify restock cart

5. Features not to be tested

5.1 Feature: Report Incident use case

This feature allows a logged-in user to report a malfunction or issue with gym equipment. It involves viewing a list of items, selecting affected items, and submitting details such as title, date, exercise context, and priority level.

Reasons for Exclusion: This feature was not developed, and the integration of this use case wasn't made to the current AlbaGym App.

Significant Combinations & Flows Excluded: The following specific functionalities and logical flows associated with this use case will **not** be tested:

- **Navigation to the Report incident window:** In the nav menu of the web won't exist a section that navigates to the Report incident window.
- **Selection Logic:** The validation requiring at least one item to be selected before proceeding won't exist in the commented use case.
- **Incident Data Submission:** The input form processing for incident details (Title, Date, Exercise, Priority, Description) and the validation of these mandatory fields.
- **Confirmation Output:** The system response showing the submitted state and the summary of reported data.

5.2 Associated Components (Classes & Entities)

All the related models and classes to the Report incident use case will exist because they are needed to test other use cases. We can see a similarity in the model to the create plan flow that will be deployed and tested in the application.

	Version: <1.3>
Test plan	Fecha: <14/12/2025>
<Test plan AlbaGym SoftwareOS>	

Reference (Link to Use Case):

- **Document:** *Use Case - AlbaGym.pdf*
- **Location:** Page 3, "Use Case: Report Incident with Item".

6. Approach

Techniques, only funtionals testing. Genereting test cases of use cases is the techniques.

Describe the overall approach to testing. For each major group of features or feature combinations, specify the approach that will ensure that these feature groups are adequately tested. Specify the major activities, techniques, and tools that are used to test the designated groups of features.

The approach should be described in sufficient detail to permit identification of the major testing tasks and estimation of the time required to do each one.

Specify the minimum degree of comprehensiveness desired. Identify the techniques that will be used to judge the comprehensiveness of the testing effort (e.g., determining which statements have been executed at least once). Specify any additional completion criteria (e.g., error frequency). The techniques to be used to trace requirements should be specified.

Identify significant constraints on testing such as test item availability, testing resource availability, and deadlines.

The testing strategy will focus exclusively on Fuctional Testing. The first techniqe employed will be Use Case Testing, where test cases are directly from the Use Case specification. This approach means we will validate the software's behaviour against requirements without inspecting the internal source code.

For each group (Plans, Purchase and Restocks) we will adopt the following approach:

We will generate test cases that strictly follow the Main Flow described in the Use Cases. For example, verifying that a user can successfully select classes and save a plan, purchase items, or complete a restock.

Then we will generate test cases based on Alternative Flows to ensure the system handles errors correctly. This includes:

Validating warnings when no classes are available.

Verify error messages when mandatory data is missing.

Checking constraints, such as attempting to buy more items that are available in stock.

To consider the testing activity complete, it will follow the next requirements:

	Version: <1.3>
Test plan	Fecha: <14/12/2025>
<Test plan AlbaGym SoftwareOS>	

Minimum Coverage: 100% of the Main flows for Create Plan, Purchase Items and Create Restock must be executed and passed.

Traceability: Every Use case has at least one corresponding Test Case.

Success Criteria: A test is Passed if the actual output matches the expected output defined in the Use Case.

The testing effort faces the following significant constraints:

Test Item Availability: Testing depends on the successful deployment of the API and Razor pages to the Azure environment.

Data dependencies: Specific test data must be preconfigured to test flows like Restock effectively.

Deadlines: Testing must be completed within the project iteration cycle.

7. Item pass/fail criteria

For pass:

- The test method executes completion without throwing any unhandled exceptions.
- All assertions defined within the test method evaluate to True
- The system state matched the expected state defined in the "Expected Results" for the scenario.
- No error or Exception logs are generated in the test output, unless explicitly tested for

For fail:

- Any assertions defined within the test method evaluate to False
- The test method is terminated prematurely by an unhandled exception
- The specific post-condition are not met
- The test exceeds the defined timeout period.

8. Suspension criteria and resumption requirements [Marcos]

8.1 Suspension Criteria

Testing activities for the AlbaGym application will be suspended immediately if any of the following conditions occur, preventing further validation of the software:

- **Authentication Failure (Blocking Pre-conditions):**
 - If the ApplicationUser login mechanism fails completely, preventing access to the system for either the "User" role or the "Administrator" role. Since all use cases require a logged-in state as a pre-condition, testing cannot proceed if the login of the respective user/administrator isn't made.

	Version: <1.3>
Test plan	Fecha: <14/12/2025>
<Test plan AlbaGym SoftwareOS>	

- **Database Unavailability (Data Retrieval Failure):**

- If the system creates a critical exception when attempting to retrieve lists of entities required for the "Main Flow" of any module. Specifically:
 - Failure to retrieve the list of classes for **Create Planning**. Also, a failure will occur if the related data for testing this use case is missing (TypeItem, ApplicationUser, PaymentMethod etc.)
 - Failure to retrieve the list of items for **Purchase Items**. Also, a failure will occur if the related data for testing this use case is missing (TypeItem, ApplicationUser, PaymentMethod etc.)
 - Failure to retrieve the list of items available for **Restock Items**. Also, a failure will occur if the related data for testing this use case is missing (TypeItem, ApplicationUser, Restock etc.)

8.2 Resumption Requirements

Testing will resume only after the specific cause of the suspension has been addressed and the following activities have been successfully completed:

- **Defect Resolution:**

- A patch or hotfix addressing the critical blocking issue (e.g., Login fix or Database connection restoration) has been deployed to the test environment.

- **Invalid data:**

- When some invalid data is placed in a mandatory form the Application can be resumed after putting the correct data in place

- **Mandatory data:**

- When some mandatory data is not placed in a mandatory form, the Application can be resumed after entering valid data in the form.

- **Wrong filtering:**

- **Re-execution:** the specific use case can be executed again when the filtering is done and it does not provide any items for the indicated conditions.

9. Test deliverables

The documents i have in my test proyect

Identify the deliverable documents. The following documents should be included:

	Version: <1.3>
Test plan	Fecha: <14/12/2025>
<Test plan AlbaGym SoftwareOS>	

- f) Test plan; yes 1
- g) Test design specifications; No
- h) Test case specifications; yes 3
- i) Test procedure specifications; No
- j) Test item transmittal reports; No
- k) Test logs; yes as many times as you run the software
- l) Test incident reports; yes inform in github by bugs
- m) Test summary reports. yes

Test input data and test output data should be identified as deliverables. Test tools (e.g., module drivers and stubs) may also be included.

l) Test plan the current document describing the scope, approach, resources and schedule.

n) Test case specifications Three sets of specifications corresponding to the main functional areas defined in the requirements:

Test cases for Create Plan of classes

Test cases for purchase items

Test cases for create restock of items.

Q) Test Logs chronological records of relevant details about the execution of test. A test log will be produced for every execution cycle performed on the software

R)Test incident reports. Any event that occurs during testing requires investigation. These will be reported and tracked via GitHub issues in the repository.

S) Test Summary Reports: A final document summarizing the result of the testing activities, including pass/fail rates and outstanding risks.

And the Test data and tools:

Test Input Data: Specific data sets required to execute the test cases, like valid User Accounts, valid Payment Methods.

Test Output Data: The actual results generated by the system, such the Database updates and Application logs.

Test Tools: The following tools are considered deliverables regarding the test environment configuration:

Visual Studio 2022: Used for running the application locally and debugging.

GitHub: Used for version control and issue/Bug tracking.

Azure App Service: Used as the hosting environment for the test items.

	Version: <1.3>
Test plan	Fecha: <14/12/2025>
<Test plan AlbaGym SoftwareOS>	

10. Testing tasks

UC_CLASSES

DtoNoClassesAvailables

```
UPDATE [dbo].[Classes]
SET [Capacity] = 0
WHERE [Id] IN (SELECT [Id] FROM [dbo].[Classes]);
```

DtoRestoreClasses

```
UPDATE [dbo].[Classes]
SET [Capacity] = 20
WHERE [Id] IN (SELECT [Id] FROM [dbo].[Classes]);
```

UC_PLAN

Dto_ClassNoCapacity

```
UPDATE [dbo].[Classes]
SET [Capacity] = 0
WHERE [Name] = N'Morning Yoga';
```

UC_PURCHASE

DtoNoItem

```
UPDATE [dbo].[Items] set QuantityAvailableForPurchase = 0 where Id = 1;
UPDATE [dbo].[Items] set QuantityAvailableForPurchase = 9 where Id = 2;
UPDATE [dbo].[Items] set QuantityAvailableForPurchase = 9 where Id = 3;
```

DtoUC9_Scen1_1

```
UPDATE [dbo].[Purchases]
SET
    [City] = N'Cuenca',
    [Country] = N'Spain',
    [Date] = CAST(GETDATE() AS DateTime),
    [Description] = N'',
    [Street] = N'Calle Mayor',
    [Total_prices] = CAST(75.00 AS Decimal(10, 2)),
    [PaymentMethodId] = 1
WHERE [Id] = 4
```

DtoUC9_Scen3_1_2

```
UPDATE [dbo].[Items] set QuantityAvailableForPurchase = 9 where Id = 1;
UPDATE [dbo].[Items] set QuantityAvailableForPurchase = 9 where Id = 2;
UPDATE [dbo].[Items] set QuantityAvailableForPurchase = 9 where Id = 3;
```

UC_RESTOCK

File_ESC3

```
INSERT INTO [dbo].[Items] ([Id], [Description], [Name], [QuantityAvailableForPurchase], [QuantityForRestock], [RestockPrice], [PurchasePrice], [ItemTypeId], [BrandId])
VALUES (12, N'Foam roller for muscle recovery and massage', N'Foam Roller', 18, 8, CAST(18.00 AS Decimal(10, 2)), CAST(25.00 AS Decimal(10, 2)), 3, 2);
```

Script base de datos software

	Version: <1.3>
Test plan	Fecha: <14/12/2025>
<Test plan AlbaGym SoftwareOS>	

```

/* =====
SCRIPT UNIFICADO DE CARGA DE DATOS (SEED)
Ordenado por dependencias para evitar errores de FK
===== */
-- 1. INSERTAR USUARIOS (AspNetUsers)
-- Nota: AspNetUsers suele tener PK string (GUID), por lo que NO requiere IDENTITY_INSERT
PRINT 'Insertando AspNetUsers...'
INSERT INTO [dbo].[AspNetUsers] ([Id], [Name], [Surname], [UserName], [NormalizedUserName], [Email], [NormalizedEmail], [EmailConfirmed], [PasswordHash], [SecurityStamp],
[ConcurrencyStamp], [PhoneNumber], [PhoneNumberConfirmed], [TwoFactorEnabled], [LockoutEnd], [LockoutEnabled], [AccessFailedCount])
VALUES (N'1', N'Pepe', N'Gomez', N'Pepe.Gomez', N'PEPE.GOMEZ', N'pepegomez@example.com', N'PEPEGOMEZ@EXAMPLE.COM', 0, NULL, NULL, NULL, N'684573945', 1, 1, NULL, 0, 0);

INSERT INTO [dbo].[AspNetUsers] ([Id], [Name], [Surname], [UserName], [NormalizedUserName], [Email], [NormalizedEmail], [EmailConfirmed], [PasswordHash], [SecurityStamp],
[ConcurrencyStamp], [PhoneNumber], [PhoneNumberConfirmed], [TwoFactorEnabled], [LockoutEnd], [LockoutEnabled], [AccessFailedCount])
VALUES (N'2', N'Alejandra', N'Jimenez', N'Alejandra.Jimenez', N'ALEJANDRA.JIMENEZ', N'Alejandrajimenez@example.com', N'ALEJANDRAJIMENEZ@EXAMPLE.COM', 0, NULL, NULL, NULL, N'684586744', 1, 1, NULL, 0, 0);

INSERT INTO [dbo].[AspNetUsers] ([Id], [Name], [Surname], [UserName], [NormalizedUserName], [Email], [NormalizedEmail], [EmailConfirmed], [PasswordHash], [SecurityStamp],
[ConcurrencyStamp], [PhoneNumber], [PhoneNumberConfirmed], [TwoFactorEnabled], [LockoutEnd], [LockoutEnabled], [AccessFailedCount])
VALUES (N'3', N'Javier', N'Hernandez', N'Javier.Hernandez', N'JAVIER.HERNANDEZ', N'Javierhernandez@example.com', N'JAVIERHERNANDEZ@EXAMPLE.COM', 0, NULL, NULL, NULL, N'645686744', 1, 1, NULL, 0, 0);
GO

-- 2. INSERTAR MARCAS (Brands)
PRINT 'Insertando Brands...'
SET IDENTITY_INSERT [dbo].[Brands] ON;
INSERT INTO [dbo].[Brands] ([Id], [Name]) VALUES (1, N'Nike');
INSERT INTO [dbo].[Brands] ([Id], [Name]) VALUES (2, N'Adidas');
INSERT INTO [dbo].[Brands] ([Id], [Name]) VALUES (3, N'Domyos');
SET IDENTITY_INSERT [dbo].[Brands] OFF;
GO

-- 3. INSERTAR TIPOS DE ITEM (ItemTypes)
PRINT 'Insertando ItemTypes...'
SET IDENTITY_INSERT [dbo].[ItemTypes] ON;
INSERT INTO [dbo].[ItemTypes] ([Id], [Name], [ClassId]) VALUES (1, N'Cardio Equipment', NULL);
INSERT INTO [dbo].[ItemTypes] ([Id], [Name], [ClassId]) VALUES (2, N'Strength Equipment', NULL);
INSERT INTO [dbo].[ItemTypes] ([Id], [Name], [ClassId]) VALUES (3, N'Accessories', NULL);
SET IDENTITY_INSERT [dbo].[ItemTypes] OFF;
GO

-- 4. INSERTAR ITEMS (Items)
-- Dependencia: Brands, ItemTypes
PRINT 'Insertando Items...'
SET IDENTITY_INSERT [dbo].[Items] ON;
INSERT INTO [dbo].[Items] ([Id], [Description], [Name], [QuantityAvailableForPurchase], [QuantityForRestock], [RestockPrice], [PurchasePrice], [ItemTypeId], [BrandId])
VALUES (7, N'Set of bands', N'Resistance band set', 20, 10, CAST(15.00 AS Decimal(10, 2)), CAST(22.00 AS Decimal(10, 2)), 1, 1);

INSERT INTO [dbo].[Items] ([Id], [Description], [Name], [QuantityAvailableForPurchase], [QuantityForRestock], [RestockPrice], [PurchasePrice], [ItemTypeId], [BrandId])
VALUES (12, N'Foam roller for muscle recovery and massage', N'Foam Roller', 18, 8, CAST(18.00 AS Decimal(10, 2)), CAST(25.00 AS Decimal(10, 2)), 3, 2);

INSERT INTO [dbo].[Items] ([Id], [Description], [Name], [QuantityAvailableForPurchase], [QuantityForRestock], [RestockPrice], [PurchasePrice], [ItemTypeId], [BrandId])
VALUES (14, N'Ideal for strength and endurance training', N'Kettlebell 10 kg', 15, 5, CAST(25.00 AS Decimal(10, 2)), CAST(35.00 AS Decimal(10, 2)), 2, 3);
SET IDENTITY_INSERT [dbo].[Items] OFF;
GO

-- 5. INSERTAR RESTOCK (Restock)
-- Dependencia: AspNetUsers
PRINT 'Insertando Restock...'
SET IDENTITY_INSERT [dbo].[Restock] ON;
INSERT INTO [dbo].[Restock] ([Id], [DeliveryAddress], [Description], [ExpectedDate], [RestockDate], [Title], [TotalPrice], [RestockResponsibleId])
VALUES (3, N'Casa', N'SinDescripcion', N'2025-10-10 00:00:00', N'2025-12-15 00:00:00', N'Restock1', CAST(100.00 AS Decimal(5, 2)), N'1');

INSERT INTO [dbo].[Restock] ([Id], [DeliveryAddress], [Description], [ExpectedDate], [RestockDate], [Title], [TotalPrice], [RestockResponsibleId])
VALUES (5, N'Universidad', N'SinDescripcion', N'2025-11-01 00:00:00', N'2025-11-13 00:00:00', N'Restock2', CAST(200.00 AS Decimal(5, 2)), N'2');

INSERT INTO [dbo].[Restock] ([Id], [DeliveryAddress], [Description], [ExpectedDate], [RestockDate], [Title], [TotalPrice], [RestockResponsibleId])
VALUES (6, N'Bar', N'SinDescripcion', N'2025-11-22 00:00:00', N'2025-12-24 00:00:00', N'Restock3', CAST(150.00 AS Decimal(5, 2)), N'3');
SET IDENTITY_INSERT [dbo].[Restock] OFF;
GO

-- 6. INSERTAR DETALLES DE RESTOCK (RestockItem)
-- Dependencia: Items, Restock
-- Nota: Esta tabla suele tener clave compuesta, por lo que NO se usa IDENTITY_INSERT usualmente
PRINT 'Insertando RestockItems...'
INSERT INTO [dbo].[RestockItem] ([ItemId], [RestockId], [Quantity], [RestockPrice]) VALUES (7, 3, 10, CAST(10.00 AS Decimal(5, 2)));
INSERT INTO [dbo].[RestockItem] ([ItemId], [RestockId], [Quantity], [RestockPrice]) VALUES (12, 5, 12, CAST(15.00 AS Decimal(5, 2)));
INSERT INTO [dbo].[RestockItem] ([ItemId], [RestockId], [Quantity], [RestockPrice]) VALUES (14, 6, 15, CAST(10.00 AS Decimal(5, 2)));
GO

PRINT 'Carga de datos finalizada correctamente.'

```

11. Environmental needs

11.1 Hardware Requirements

The testing process requires workstations capable of supporting local development environments and web-based execution.

- **Test Stations:** Standard PCs or Laptops with sufficient processing power (min. 4GB RAM, Quad-core CPU) to run the application locally if needed.
- **Network:** A stable internet connection is necessary to access **Azure** cloud resources and the **GitHub** repository.

	Version: <1.3>
Test plan	Fecha: <14/12/2025>
<Test plan AlbaGym SoftwareOS>	

11.2 Software Requirements

The following software components are required to build, deploy, and execute the tests for the AlbaGym application:

- **Integrated Development Environment (IDE):**
 - **Visual Studio Code:** The primary tool for code review, debugging, and executing automated test scripts.
- **Cloud Infrastructure:**
 - **Microsoft Azure:**
 - **Azure SQL Database:** To host the test database containing entity tables such as Class, Item and PaymentMethod.
 - **Azure App Service:** To host the deployed test version of the web application.
- **Version Control:**
 - **GitHub:** Required for source code management, accessing the latest build of the application, and potentially for tracking issues.
- **Web Browsers:**
 - Recent versions of **Google Chrome** or **Microsoft Edge** to verify the web interface during the execution of Main Flows (e.g., "Create Planning" or "Purchase Items").
- **Other softwares:**
 - SQL to local testing
 - Selenium
 - JMeter

11.3 Mode of Usage & Security

- **Mode of Usage:**
 - Testing will be performed in a **Client-Server** mode. Testers will access the application via a browser (Client) which communicates with the backend hosted on Azure (Server). Also is possible to test in local thanks to SQL and the data in the testing device.
- **Security:**
 - **Role-Based Access:** The test environment must provide pre-configured

	Version: <1.3>
Test plan	Fecha: <14/12/2025>
<Test plan AlbaGym SoftwareOS>	

accounts with specific roles to satisfy pre-conditions:

- **"User" Role:** Required for testing *Create Planning* and *Purchase Items*.
- **"Administrator" Role:** Required for testing *Restock Items*.

	Version: <1.3>
Test plan	Fecha: <14/12/2025>
<Test plan AlbaGym SoftwareOS>	

12. Responsibilities

Identify the groups responsible for managing, designing, preparing, executing, witnessing, checking, and resolving. In addition, identify the groups responsible for providing the test items identified in 4.2.3 and the environmental needs identified in 4.2.

These groups may include the developers, testers, operations staff, user representatives, technical support staff, data administration staff, and quality support staff.

Test Management Group:

Role Test manager

Responsibilities:

Managing oversee the implementation of this Test Plan and monitor the schedule.

Checking review test logs and ensure test cases are traced back to user stories.

Verify the final results of the Test Summary Report.

Test Design and Execution Group

Role Testers/ Test Designers.

Responsibilities:

Designing: Create specific test cases based on the Use cases.

Set Up the test data required before execution.

Perform the manual testing on the Razor Web Application and record the results in the Test Logs.

Assigned: All the members.

Development Group:

Role Developers.

Responsibilities:

Analyze and fix the bugs reported during testing via GitHub Issues.

Ensure the latest version of the Web API and razor is compiled and available in the repository,

Assigned: All the members.

Technical Support Group

Role Environment Support

Responsibilities:

Manage the Azure App Service, Database connectivity and GitHub repository.

	Version: <1.3>
Test plan	Fecha: <14/12/2025>
<Test plan AlbaGym SoftwareOS>	

13. Staffing and training needs[no]

Specify test staffing needs by skill level. Identify training options for providing necessary skills.

14. Schedule

Include test milestones identified in the software project schedule as well as all item transmittal events.

Define any additional test milestones needed. Estimate the time required to do each testing task. Specify the schedule for each testing task and test milestone. For each testing resource (i.e., facilities, tools, and staff), specify its periods of use.

When we are going to start and finish the activities.

Name task	Start date	End Date
Creation of the models and initial data (sprint 1)	19/09/2025	3/10/2025
Creation of the methods and their respective test (sprint 2)	4/10/2025	14/11/2025
Creation of the web app, the UI and their respective test (sprint 3)	15/11/2025	14/12/2025

15. Risks and contingencies

Trheats

Identify the high-risk assumptions of the test plan. Specify contingency plans for each (e.g., delayed delivery of test items might require increased night shift scheduling to meet the delivery date).

Risk: delayed delivery of software builds for testing.

Priorize the execution of critical functionalities first. If the time is insufficient, reduce the scope.

Risk Test environment instability or data corruption.

	Version: <1.3>
Test plan	Fecha: <14/12/2025>
<Test plan AlbaGym SoftwareOS>	

Perform manual database restores to known valid state from backups. If automation fails due to data issues, temporally switch to manual execution for the affected scenarios.

16. Approvals

Logins

Email: Adrian.Sevilla@alu.uclm.es

Password:Password123!

Email: test@gmail.com

Password:Password123!